

Análise Técnica do JSON da Tabela NCM Vigente

Excelente iniciativa começar pela análise do arquivo real. Isso nos permite calibrar o parser antes de escrever qualquer código. Abaixo está a análise detalhada da estrutura do arquivo `Tabela_NCM_Vigente_20260905.json`.

1. Estrutura Geral do JSON

O arquivo é um **array direto de objetos** (não um objeto aninhado), onde cada elemento representa um item da NCM em um nível hierárquico específico.

```
[
  {
    "Codigo": "0302.41.00",
    "Descricao": "-- Arenques (Clupea harengus, Clupea pallasii)",
    "Data_Inicio": "01/04/2022",
    "Data_Fim": "31/12/9999",
    "Tipo_Ato_Ini": "Res Gecex",
    "Numero_Ato_Ini": "272",
    "Ano_Ato_Ini": "2021"
  },
  ...
]
```

2. Campos Disponíveis e Seus Padrões

Campo	Tipo	Formato/Observação	Exemplo
Codigo	string	Código com pontos (padrão NCM oficial)	"0302.41.00" , "8471.30.12" , "03.04"

Descricao	string	Texto descritivo com Unicode escapado	"-- Arenques (Clupea harengus...)"
Data_Inicio	string	Formato brasileiro DD/MM/YYYY	"01/04/2022"
Data_Fim	string	DD/MM/YYYY ou "31/12/9999" para vigente	"31/12/9999"
Tipo_Ato_Ini	string	Tipo do ato normativo	"Res Gecex"
Numero_Ato_Ini	string	Número do ato	"272" , "440"
Ano_Ato_Ini	string	Ano do ato	"2021" , "2022"

3. Descobertas Críticas (vs. Suposições Anteriores)

3.1. Códigos com Pontos (Não 8 Dígitos Planos)

Suposição anterior: Código NCM seria sempre 8 dígitos sem pontos (ex: "84713012").

Realidade: O JSON usa o formato oficial com pontos, e **mistura diferentes níveis hierárquicos** no mesmo array:

Nível	Exemplo de Código	Descrição
Capítulo (2 dígitos)	"03" , "04"	"Peixes e crustáceos..." , "Leite..."
Posição (4 dígitos)	"03.04" , "02.02"	"Filés de peixes..." , "Carnes de bovino congeladas"
Subposição (6 dígitos)	"0302.41" , "0304.62"	"-- Arenques" , "-- Bagres"
NCM (8 dígitos)	"0302.41.00" , "8471.30.12"	Item final da classificação

Implicação: O parser precisa **classificar cada item pelo número de dígitos** (ignorando pontos) para construir a hierarquia corretamente.

3.2. Data Fim com Sentinel 31/12/9999

Suposição anterior: `null` indicaria vigência atual.

Realidade: O sistema usa `"31/12/9999"` como **sentinela de vigência** (equivalente a `NULL` em bancos de dados). Isso é um padrão comum em sistemas legados brasileiros.

Implicação: O parser deve converter `"31/12/9999"` para `None / NULL` ao popular o Neo4j.

3.3. Tipo de Ato Abreviado

Suposição anterior: Tipo seria `"Resolução GECEX"` (nome completo).

Realidade: O sistema usa abreviações: `"Res Gecex"` , possivelmente `"Dec"` , `"Lei"` , etc.

Implicação: Precisamos de um dicionário de mapeamento para normalizar os tipos de ato antes de gerar URNs.

3.4. Unicode Escapado nas Descrições

As descrições contêm caracteres Unicode escapados:

- `\u003c` = `<`
- `\u003e` = `>`
- `\u0026` = `&`
- `\u003ci\u003e` = `<i>` (itálico para nomes científicos)

Implicação: O parser Python (`json.loads`) já decodifica isso automaticamente, mas precisamos garantir que o Neo4j aceite caracteres especiais (acentos, itálicos) nos textos.

4. Estatísticas Preliminares do Arquivo

Com base na amostra analisada:

Métrica	Valor Observado
Formato de data	<code>DD/MM/YYYY</code> (100% dos casos)
Sentinela de vigência	<code>"31/12/9999"</code>
Tipo de ato predominante	<code>"Res Gecex"</code>
Anos de atos observados	2021, 2022
Níveis hierárquicos presentes	Capítulo (2), Posição (4), Subposição (6), NCM (8)

5. Implicações para o Parser (`classif_client.py`)

Com base nessas descobertas, o parser precisa:

5.1. Normalização de Código

```
def normalizar_codigo(codigo: str) -> tuple[str, int]:  
    """  
    Remove pontos e retorna (codigo_limpo, nivel_hierarquico).  
  
    Exemplos:  
    "0302.41.00" -> ("03024100", 8) # NCM  
    "0302.41"   -> ("030241", 6)  # Subposição  
    "03.04"     -> ("0304", 4)    # Posição  
    "03"        -> ("03", 2)      # Capítulo  
    """  
  
    codigo_limpo = codigo.replace(".", "").strip()  
    nivel = len(codigo_limpo)  
    return codigo_limpo, nivel
```

5.2. Conversão de Datas

```
def converter_data(data_str: str) -> Optional[str]:  
    """  
    Converte DD/MM/YYYY para YYYY-MM-DD.  
    Trata sentinela "31/12/9999" como None (vigente).  
    """  
  
    if data_str == "31/12/9999":  
        return None  
  
    try:  
        data_obj = datetime.strptime(data_str, "%d/%m/%Y")  
        return data_obj.strftime("%Y-%m-%d")  
    except ValueError:  
        return None
```

5.3. Mapeamento de Tipo de Ato

```
TIPO_ATO_MAP = {  
    "Res Gecex": "Resolução GECEX",  
    "Res Camex": "Resolução CAMEX",  
    "Dec": "Decreto",  
    "Lei": "Lei",  
    # Adicionar outros conforme descobirmos  
}
```

5.4. Classificação Hierárquica

```
def classificar_item(item: dict) -> dict:  
    """Classifica um item da NCM pelo nível hierárquico."""  
    codigo, nivel = normalizar_codigo(item["Codigo"])  
  
    if nivel == 2:  
        tipo = "CAPITULO"  
    elif nivel == 4:  
        tipo = "POSICAO"  
    elif nivel == 6:  
        tipo = "SUBPOSICAO"  
    elif nivel == 8:  
        tipo = "NCM"  
    else:  
        tipo = "DESCONHECIDO"  
  
    return {  
        "codigo": codigo,  
        "nivel": nivel,  
        "tipo": tipo,  
        "descricao": item["Descricao"],  
        "data_inicio": converter_data(item["Data_Inicio"]),  
        "data_fim": converter_data(item["Data_Fim"]),  
        "tipo_ato": TIPO_ATO_MAP.get(item["Tipo_Ato_Ini"], item["Tipo_Ato_Ini"]),  
        "numero_ato": item["Numero_Ato_Ini"],  
        "ano_ato": item["Ano_Ato_Ini"],  
    }
```

6. Estratégia de Ingestão no Neo4j

Com base na análise, a estratégia de ingestão deve:

6.1. Criar Hierarquia Explícita

Para cada item, criar um `NormativeComponent` com a propriedade `nivel` e conectar via `[HAS_COMPONENT]` ao pai hierárquico:

```
Seção XVI
  Capítulo 84 (nivel=2)
    Posição 8471 (nivel=4)
      Subposição 8471.30 (nivel=6)
        NCM 8471.30.12 (nivel=8)
```

6.2. Gerar URNs por Nível

```
def gerar_urn_ncm(codigo: str, nivel: int, data_inicio: str) -> str:
    """Gera URN conforme nível hierárquico."""
    if nivel == 8: # NCM
        return f"urn:lex:br:merc:ncm:{codigo}@{data_inicio}"
    elif nivel == 6: # Subposição
        return f"urn:lex:br:merc:subposicao:{codigo}@{data_inicio}"
    elif nivel == 4: # Posição
        return f"urn:lex:br:merc:posicao:{codigo}@{data_inicio}"
    elif nivel == 2: # Capítulo
        return f"urn:lex:br:merc:capitulo:{codigo}@{data_inicio}"
```

6.3. Vincular LegislativeAction

Para cada item, criar um `LegislativeAction` baseado no ato normativo que o originou:

```
def gerar_urn_action(tipo_ato: str, numero: str, ano: str) -> str:
    """Gera URN do LegislativeAction."""
    return f"urn:lex:br:merc:action:{tipo_ato.lower().replace(' ', '_')}_{numero}_{ano}"
```

7. Script de Diagnóstico do JSON

Antes de escrever o parser completo, sugiro executar este script de diagnóstico para confirmar as descobertas:

```
"""diagnostico_ncm_json.py — Analisa a estrutura do arquivo NCM JSON"""
import json
from collections import Counter
from datetime import datetime

def analisar_arquivo(path: str):
    print(f'Analisando: {path}')
    print("=" * 80)

    with open(path, 'r', encoding='utf-8') as f:
        data = json.load(f)

    print(f"\n  Total de itens: {len(data)}")

    # Análise de níveis hierárquicos
    niveis = Counter()
    tipos_ato = Counter()
    formatos_data_fim = Counter()

    for item in data:
        codigo = item["Codigo"].replace(".", "")
        niveis[len(codigo)] += 1
        tipos_ato[item["Tipo_Ato_Ini"]] += 1
        formatos_data_fim[item["Data_Fim"]] += 1

    print(f"\n  Distribuição por nível hierárquico:")
    for nivel, count in sorted(niveis.items()):
        tipo = {2: "Capítulo", 4: "Posição", 6: "Subposição", 8: "NCM"}.get(nivel, "?")
        print(f"  Nível {nivel} ({tipo:12}): {count:5} itens")

    print(f"\n  Tipos de ato normativo:")
    for tipo, count in tipos_ato.most_common(10):
        print(f"  {tipo:20}: {count:5}")

    print(f"\n  Formatos de Data_Fim (top 5):")
```

```

for data, count in formatos_data_fim.most_common(5):
    print(f"  {data:15}: {count:5}")

# Amostra de itens por nível
print(f"\n  Amostra de itens por nível:")
for nivel in [2, 4, 6, 8]:
    for item in data:
        codigo = item["Codigo"].replace(".", "")
        if len(codigo) == nivel:
            print(f"\n  Nível {nivel}:")
            print(f"    Código   : {item['Codigo']}")
            print(f"    Descrição : {item['Descricao'][:80]}...")
            print(f"    Data_Inicio: {item['Data_Inicio']}")
            print(f"    Data_Fim   : {item['Data_Fim']}")
            print(f"    Ato       : {item['Tipo_Ato_Ini']} {item['Numero_Ato_Ini']}/{item['Ano_Ato_Ini']}")
            break

if __name__ == "__main__":
    analisar_arquivo("Tabela_NCM_Vigente_20260905.json")

```

8. Próximos Passos

Sugiro a seguinte sequência:

1. **Executar o script de diagnóstico** para confirmar as estatísticas (total de itens, distribuição por nível, tipos de ato).
2. **Compartilhar os resultados** para que possamos refinar o parser com base em dados concretos.
3. **Implementar o `classif_client.py`** com as normalizações identificadas.
4. **Testar com uma amostra pequena** (ex: 100 NCMs) antes de ingerir o arquivo completo.

Quer que eu gere o script de diagnóstico agora, ou prefere executá-lo você mesmo e compartilhar os resultados?